Faculty of Computers and Artificial Intelligence
Cairo University
CS213: Programming II (Object Oriented Programming)
Year 2026-2027, First Semester

Assignment 2 - Part 1

Task 1: OOP Design of the Vole Machine Simulator

Course instructor: Dr. Mohammad El-Ramly

Team members

[name] - ID: [ID]; Email: [email]; Section: [section]

[name] - ID: [ID]; Email: [email]; Section: [section]

[name] - ID: [ID]; Email: [email]; Section: [section]

[name] - ID: [ID]; Email: [email]; Section: [section]

Team leader: [name] Phone: [phone] Date: [date]

Work breakdown table (who did what)

[name]: [tasks]

[name]: [tasks]

[name]: [tasks]

[name]: [tasks]

TO BE COMPLETED BY THE TEAM: fill in the names, IDs, emails, section, leader phone, date and the work breakdown. Rename this file to CS213_A2_Part1_SectionNum_YourID_YourID_YourID_YourID.pdf with your real section and IDs.

1. Introduction: the Vole machine

The Vole machine is a fictitious, very small computer used to teach how a real computer executes programs. The simulator we are designing imitates this machine: it loads a program into a simulated memory and executes it instruction by instruction.

1.1 Architecture

Main memory: 256 cells with addresses 00 to FF. Each cell holds 8 bits (2 hex digits).

Registers: 16 general purpose registers numbered 0 to F. Each holds 8 bits.

Program Counter (PC): Holds the address of the next instruction to fetch.

Instruction Register (IR): Holds the instruction currently being executed (16 bits = 4 hex digits).

Instruction format: Every instruction is 16 bits and occupies two consecutive memory cells: the first hex digit is the op-code and the other three are the operand (R, S, T or X, Y).

Screen: Memory cell FF is mapped to the screen: storing a value in FF (instruction 3RFF) prints it on the screen.

Program start address: Must be an even address below FF (00, 02, 04, ...) because instructions take two cells.

1.2 Machine cycle

Fetch: read the two cells at address PC into IR and add 2 to PC. Decode: validate the op-code (1 to C) and split IR into its fields. Execute: perform the operation. A jump (BRXY) executes by copying XY into PC. The cycle repeats until the Halt instruction (C000) is executed.

1.3 Instruction set (2026/27 version)

Op 1 - Operand: RXY; Meaning: Load register R from memory cell XY

Op 2 - Operand: RXY; Meaning: Load register R with the value XY

Op 3 - Operand: RXY; Meaning: Store register R into memory cell XY

Op 3 - Operand: RFF; Meaning: Store register R to FF, which writes to the screen

Op 4 - Operand: 0RS; Meaning: Move register R to register S

Op 5 - Operand: RST; Meaning: R = S + T as two's complement integers

Op 6 - Operand: RST; Meaning: R = S + T as floating-point values

Op 7 - Operand: RST; Meaning: R = S OR T

Op 8 - Operand: RST; Meaning: R = S AND T

Op 9 - Operand: RST; Meaning: R = S XOR T

Op A - Operand: R0X; Meaning: Rotate register R right X times (circular)

Op B - Operand: RXY; Meaning: If register R equals register 0, set PC = XY (jump)

Op C - Operand: 000; Meaning: Halt

1.4 What the simulator must do

Offer a menu; load a program from a file at an even start address below FF; let the user overwrite a register or memory cell; set PC to the program start; display memory as a 16x16 matrix with IR and the 16 registers; fetch, validate, decode and execute step by step while updating PC; let the user run all at once or step by step (showing the machine state at the end, or after each step followed by Enter); finally ask whether to exit or load another program.

2. Initial design

2.1 Classes and responsibilities

Simulator: User interface: shows the menu and reads choices, and calls the other classes. Does no machine logic itself.

FileLoader: Reads a program file, ignores comment lines (starting with ;) and empty lines, keeps only the first 4 hex digits of each line, validates them and writes them into memory from a chosen even start address.

Machine: Owns the CPU, Memory and Screen. Starts the program and runs it all at once or one step at a time, then displays the machine state.

CPU: Owns PC, IR and the registers. Implements the fetch, decode, execute cycle and updates PC.

Memory: Stores the 256 cells, with validated read and write and a 16x16 matrix display.

Registers: Stores the 16 general purpose registers, with get, set and display.

Instruction: Represents one decoded instruction (op-code and operand fields) and checks that it is valid.

ALU: Performs the arithmetic and logic operations: integer add, floating-point add, OR, AND, XOR and rotate.

Screen: Receives values written to address FF and displays them.

2.2 Attributes and methods

Simulator
Attributes:
-machine : Machine
-loader : FileLoader
Methods:
+showMenu()
+loadProgram()
+editRegister()
+editMemoryCell()
+chooseRunMode()
+askExitOrLoad()

FileLoader
Attributes:
-filePath : string
Methods:
+readFile(path) : string[]
+cleanLine(line) : string
+isValidInstruction(s) : bool
+loadToMemory(mem, start)

Machine
Attributes:
-cpu : CPU
-memory : Memory
-screen : Screen
Methods:
+loadProgram(list, start)
+setPC(addr)
+runAll()
+runStep()
+displayState()

CPU
Attributes:
-pc : int
-ir : string
-halted : bool
-registers : Registers
-alu : ALU
Methods:
+fetch(mem)
+decode() : Instruction
+execute(inst, mem, scr)
+step(mem, scr)
+isHalted() : bool
+displayState()

Memory
Attributes:
-cells : string[256]
Methods:
+read(addr) : string
+write(addr, value)
+isValidAddress(addr) : bool
+display()

Registers
Attributes:
-regs : string[16]
Methods:
+get(index) : string
+set(index, value)
+display()

Instruction
Attributes:
-raw : string
-opcode : char
-r, -s, -t : int
-x, -y : int
Methods:
+parse(ir)
+isValid() : bool
+getOpcode() : char

ALU
Attributes:
(none)
Methods:
+addInt(a, b)
+addFloat(a, b)
+bitOr(a, b)
+bitAnd(a, b)
+bitXor(a, b)
+rotateRight(v, n)

Screen
Attributes:
-buffer : string
Methods:
+write(value)
+display()
+clear()

2.3 Relations between classes

Simulator -> Machine - Type: Association; Why: The simulator uses one machine.

Simulator -> FileLoader - Type: Association; Why: The simulator asks the loader to read program files.

FileLoader ..> Memory - Type: Dependency; Why: Writes the program into memory, but does not own it.

Machine *-- CPU, Memory, Screen - Type: Composition; Why: The machine is made of them and they do not exist without it.

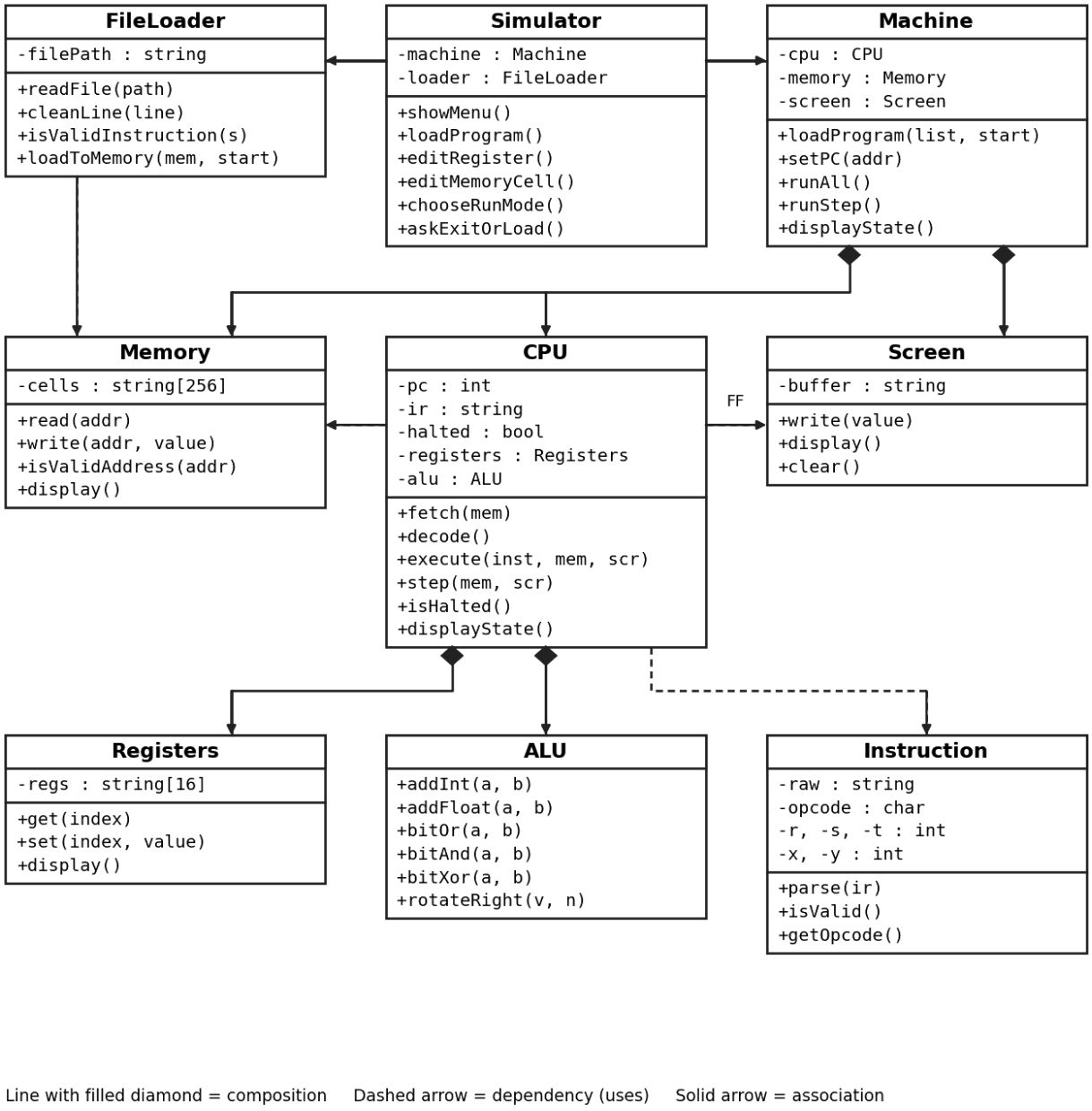
CPU *-- Registers, ALU - Type: Composition; Why: The registers and the ALU are internal parts of the CPU.

CPU ..> Instruction - Type: Dependency; Why: Creates an Instruction at each decode step.

CPU -> Memory - Type: Association; Why: Reads instructions and data and writes data.

CPU -> Screen - Type: Association; Why: Writes to the screen when the address is FF.

2.4 UML class diagram



Line with filled diamond = composition Dashed arrow = dependency (uses) Solid arrow = association

TO BE COMPLETED BY THE TEAM: draw the diagram from the PlantUML code below at https://www.plantuml.com/ and replace the picture above with that official rendering if your instructor expects it.

2.5 PlantUML code

```
@startuml
skinparam classAttributeIconSize 0
class Simulator {
 -machine : Machine
 -loader : FileLoader
 +showMenu() : void
 +loadProgram() : void
 +editRegister() : void
 +editMemoryCell() : void
 +chooseRunMode() : void
 +askExitOrLoad() : void
}
class FileLoader {
 -filePath : string
 +readFile(path : string) : string[]
 +cleanLine(line : string) : string
 +isValidInstruction(s : string) : bool
 +loadToMemory(mem : Memory, start : int) : void
}
class Machine {
 -cpu : CPU
 -memory : Memory
 -screen : Screen
 +loadProgram(list : string[], start : int) : void
 +setPC(addr : int) : void
 +runAll() : void
 +runStep() : void
 +displayState() : void
}
class CPU {
 -pc : int
 -ir : string
 -halted : bool
 -registers : Registers
 -alu : ALU
 +fetch(mem : Memory) : void
 +decode() : Instruction
 +execute(inst : Instruction, mem : Memory, scr : Screen) : void
 +step(mem : Memory, scr : Screen) : void
 +isHalted() : bool
 +displayState() : void
}
class Memory {
 -cells : string[256]
 +read(addr : int) : string
 +write(addr : int, value : string) : void
 +isValidAddress(addr : int) : bool
 +display() : void
}
class Registers {
 -regs : string[16]
 +get(index : int) : string
 +set(index : int, value : string) : void
 +display() : void
}
class Instruction {
 -raw : string
 -opcode : char
 -r : int
 -s : int
 -t : int
 -x : int
 -y : int
 +parse(ir : string) : void
 +isValid() : bool
 +getOpcode() : char
}
class ALU {
 +addInt(a : int, b : int) : int
 +addFloat(a : float, b : float) : float
```

```
 +bitOr(a : int, b : int) : int
 +bitAnd(a : int, b : int) : int
 +bitXor(a : int, b : int) : int
 +rotateRight(v : int, n : int) : int
}
class Screen {
 -buffer : string
 +write(value : string) : void
 +display() : void
 +clear() : void
}
Simulator --> Machine
Simulator --> FileLoader
FileLoader ..> Memory : loads into
Machine *-- CPU
Machine *-- Memory
Machine *-- Screen
CPU *-- Registers
CPU *-- ALU
CPU ..> Instruction : creates
CPU --> Memory : reads/writes
CPU --> Screen : writes at FF
@enduml
```

3. AI designs

For each of the three different AIs include: the full prompts and the full conversation, the PlantUML code of the design it produced, and the UML class diagram drawn from that code.

3.1 AI number 1

TO BE COMPLETED BY THE TEAM: AI name and version; full conversation (your questions and why it designed it this way); PlantUML code; UML diagram picture.

3.2 AI number 2

TO BE COMPLETED BY THE TEAM: AI name and version; full conversation (your questions and why it designed it this way); PlantUML code; UML diagram picture.

3.3 AI number 3

TO BE COMPLETED BY THE TEAM: AI name and version; full conversation (your questions and why it designed it this way); PlantUML code; UML diagram picture.

3.4 Comparison of the three designs

TO BE COMPLETED BY THE TEAM: how similar or different the three designs are and which AI produced the best result.

4. Final design

TO BE COMPLETED BY THE TEAM: take the best of the three designs, give it to another AI, and include the full conversation and prompts used to enhance it, the final PlantUML code, the final UML diagram, and why it is the best design.

4.1 Experience and what we learned

TO BE COMPLETED BY THE TEAM: describe your experience with the AIs and what you learned.

5. Examples: two Vole programs

Both programs follow the file format of the assignment. They use 32FF / 34FF to print on the screen, because the 2026/27 instruction table maps the screen to address FF (the older sample file used 3200).

5.1 Program 1: perimeter of a rectangle

Computes 2 x (5 + 3) = 16, which is 10 in hex, and prints it.

```
; ------------------------------------------------
; Vole Machine Program 1
; Purpose: Calculate the perimeter of a rectangle
; (width = 5, height = 3) and display it: 2*(5+3) = 16 = 10h
; ------------------------------------------------
2105 ; R1 = 05 (width)
2203 ; R2 = 03 (height)
5312 ; R3 = R1 + R2
5433 ; R4 = R3 + R3 (perimeter)
34FF ; Store R4 to FF (screen)
C000 ; Halt
```

Step 1 - Instruction: 2105; Effect: R1 = 05; Result: R1 = 05

Step 2 - Instruction: 2203; Effect: R2 = 03; Result: R2 = 03

Step 3 - Instruction: 5312; Effect: R3 = R1 + R2; Result: R3 = 08

Step 4 - Instruction: 5433; Effect: R4 = R3 + R3; Result: R4 = 10

Step 5 - Instruction: 34FF; Effect: screen = R4; Result: screen shows 10

Step 6 - Instruction: C000; Effect: halt; Result: program stops

5.2 Program 2: sum of 1 to 5 using a loop

Adds 1+2+3+4+5 = 15, which is 0F in hex, and prints it. R0 holds the limit N (change the first line to change N). Instruction B008 compares R0 with itself, so it always jumps back (unconditional jump). The program must be loaded at address 00 because the jump addresses are absolute.

```
; ---------------------------------------------
; Vole Machine Program 2
; Purpose: Calculate 1+2+3+4+5 using a loop and display
; the result: 15 = 0F. Change the first line to change N.
; NOTE: load this program at address 00 (jumps are absolute).
; ---------------------------------------------
2005 ; 00: R0 = 05 (N, the limit)
2100 ; 02: R1 = 00 (counter i)
2200 ; 04: R2 = 00 (sum)
2301 ; 06: R3 = 01 (constant one)
5113 ; 08: R1 = R1 + R3 (i = i + 1)
5221 ; 0A: R2 = R2 + R1 (sum = sum + i)
B110 ; 0C: if R1 == R0 jump to 10 (end of loop)
B008 ; 0E: R0 == R0 always, so jump back to 08
32FF ; 10: Store R2 to FF (screen)
C000 ; 12: Halt
```

Loop pass 1 - R1 (i): 01; R2 (sum): 01; B110 (R1 == R0 ?): no, continue; B008 jumps to 08

Loop pass 2 - R1 (i): 02; R2 (sum): 03; B110 (R1 == R0 ?): no, jump back

Loop pass 3 - R1 (i): 03; R2 (sum): 06; B110 (R1 == R0 ?): no, jump back

Loop pass 4 - R1 (i): 04; R2 (sum): 0A; B110 (R1 == R0 ?): no, jump back

Loop pass 5 - R1 (i): 05; R2 (sum): 0F; B110 (R1 == R0 ?): yes, PC = 10; screen shows 0F; then halt

6. Video link

TO BE COMPLETED BY THE TEAM: paste the link to your ~10 minute video discussing the three designs, which AI produced the best results, and how similar or different they are.